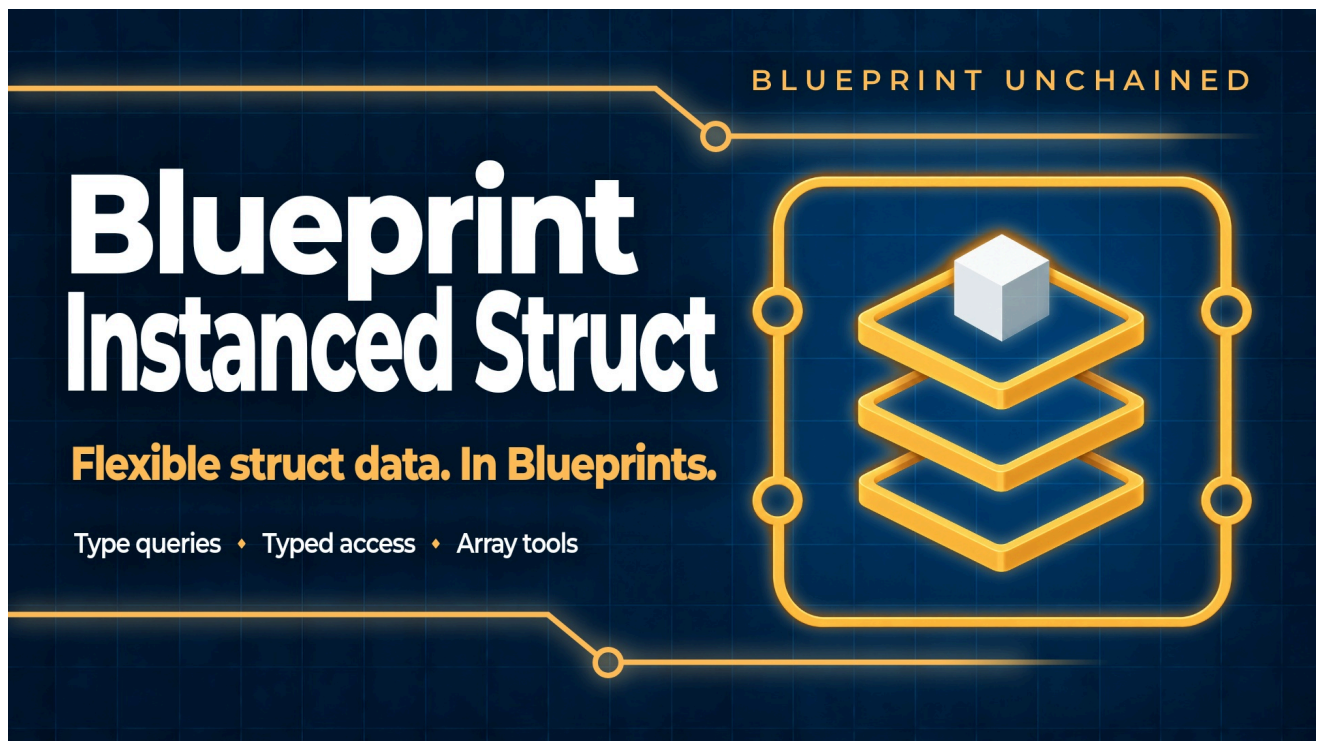


# Blueprint Instanced Struct

---



Author: **Tomasz Klin**

Version: **1.0.0**

---

## Contents

### Introduction

### Getting started

### Core concepts

- One value, one current type
- Values are copied
- Native inheritance and Blueprint structs

### Creating values

- Make Instanced Struct
- Make Default Of Type

### Reading and writing values

- Choose a type on Get and Set
- Get Struct Value
- Set Struct Value

### Inspecting and comparing

### Saving and loading text

### Arrays by type

### Recipes

- Build an event queue with different payloads
- Store different inventory data in one list
- Copy a value through text
- Create a typed default from a data-driven selection

### Troubleshooting

- Get Struct Value takes the failure path
- The wildcard pin has the wrong type
- An array will not connect to a wildcard struct pin
- Parsed text fails after a type was renamed
- I see similar nodes elsewhere in the menu



# Introduction

---

Blueprint Instanced Struct helps you read, change and find values that use different struct types. For example, you can keep weapon and potion data in the same inventory array.

Unreal Engine's **Instanced Struct** type holds one struct value at a time. You can replace it with a value of another struct type while the game runs. An array of Instanced Struct values can hold a different type in each entry.

This plugin adds nodes to choose types, check values, search arrays and convert data to text and back. It uses the engine's existing `FInstancedStruct` type. You can use it entirely in Blueprint.

# Getting started

---

1. Enable **Blueprint Instanced Struct** in **Edit > Plugins**. Restart the editor if prompted.
2. Create a variable of type **Instanced Struct**.
3. Connect your struct to **Make Instanced Struct** and assign the result to the variable.
4. Add **Get Struct Value** and choose the expected struct type in the node's picker.
5. Connect your Instanced Struct value. Use the success output to work with the result.

The failure output runs if the value is empty or has the wrong type. Use **Set Struct Value** when you want to replace the value already stored in the variable.

Find all nodes under **Instanced Struct** in the Blueprint menu. You can use them without placing an actor or changing Project Settings.

# Core concepts

---

## One value, one current type

An Instanced Struct is either empty or contains one complete struct value and its type. Setting a new value replaces the previous value.

Use **Is Valid** to check whether it contains a struct. Use **Get Script Struct** to get its type.

## Values are copied

Make and Set copy a struct into the Instanced Struct value. Get copies it out. Changing the output from Get does not change the stored value. Call **Set Struct Value** with your edited struct to store the change.

## Native inheritance and Blueprint structs

C++ structs can have parent and child types. **Is Struct Type** accepts a child value when you check for its parent type. **Get Struct Value** can read the fields defined by that parent.

Structs made in Blueprint cannot inherit from each other. Their types must match exactly.

# Creating values

---

## Make Instanced Struct

Connect one struct value to **Make Instanced Struct**. The input pin takes the connected struct's type, and the result holds a copy of its value. This kind of pin is called a wildcard pin.

## Make Default Of Type

Use **Make Default Of Type** when you choose the type while the game runs and have no value to copy. It creates a value with that struct's defaults.

The node is pure, so it needs no execution wire and can be used while building an array. An empty or invalid **Struct Type** input produces an empty Instanced Struct value.

# Reading and writing values

---

## Choose a type on Get and Set

**Get Struct Value** and **Set Struct Value** show a type picker on the node. Choose a struct to set the pin's type. You can then split the pin into fields or connect it to other nodes.

The picker lists structs that Blueprint variables can use. A connected struct pin sets the type automatically and overrides the picker. Disconnect that pin before choosing another type.

This choice is made while editing the graph. It determines the pin type when the Blueprint compiles.

## Get Struct Value

Get copies the stored value into its typed output. The success execution output runs if the stored type is compatible with the selected type. An empty value or a different, incompatible type runs the failure output. Use the copied value only after success.

## Set Struct Value

Set replaces the stored value with a copy of the connected struct. The input accepts one struct. For an array, store Instanced Struct values as its entries and use the array nodes below.

# Inspecting and comparing

---

- **Is Valid (Instanced Struct)** checks whether a value contains a struct.
- **Get Script Struct** returns the stored type, or none if the value is empty.
- **Is Struct Type** checks the type, including parent types for C++ structs.
- **Reset (Instanced Struct)** clears the stored value.
- **Equal (Instanced Struct)** checks for the same type and equal field values.
- **Not Equal (Instanced Struct)** returns the opposite result.
- **To String (Instanced Struct)** creates readable text for logs and debugging.

To save a value as text and read it back later, use **Export To String** and **Parse From String**.

# Saving and loading text

---

**Export To String** converts the struct type and field values to text. Store that text in your own save or configuration system. Use **Parse From String** to create an Instanced Struct from it.

Check the Parse result before using the value. Parsing fails if the text is invalid or Unreal Engine cannot find the stored struct type.

If you rename a struct or change its fields, existing saved text may need an update. Plan how older saves will be read when you make those changes.

# Arrays by type

---

These nodes take an array of Instanced Struct values and a **Struct Type**:

| Node               | Result                                                                              |
|--------------------|-------------------------------------------------------------------------------------|
| Find First Of Type | Returns the first matching value and its index. The index is -1 if no match exists. |
| Filter By Type     | Returns all matching values in their original order.                                |
| Contains Type      | Checks whether at least one matching entry exists.                                  |
| Count Of Type      | Returns the number of matching entries.                                             |
| Remove All Of Type | Removes matching entries from the input array and returns how many were removed.    |

Type checks follow the inheritance rules described above. Use **Contains Type** when you only need a yes-or-no answer. Use **Filter By Type** when you need the matching values.

# Recipes

---

These examples show how to use the nodes in common Blueprint tasks.

## Build an event queue with different payloads

A payload is the data carried by an event. For example, a damage event can carry a damage amount and a target, while a healing event carries a healing amount.

1. Create a struct for each event type.
2. Put event values into an array of Instanced Struct values.
3. Use **Is Struct Type** to choose the logic for each event.
4. Use **Get Struct Value** to read the event's data.

Use a C++ parent struct if several event types need to match the same parent type check.

## Store different inventory data in one list

Keep shared fields, such as an item ID, in your inventory entry. Store each item's specific data in an Instanced Struct.

Weapon, potion and quest-item structs can share one array. Each struct only needs the fields used by that kind of item.

## Copy a value through text

1. Use **Export To String** on the value.
2. Store or send the resulting text through your own system.
3. Use **Parse From String** to read the text.
4. Check the Parse result, then read the value with **Get Struct Value**.

## Create a typed default from a data-driven selection

Store a **Struct Type** in your project data and pass it to **Make Default Of Type**. The node creates a value with that type's defaults.

Use this when a designer chooses the struct type in data and the game creates the value later.

# Troubleshooting

---

## Get Struct Value takes the failure path

Check whether the value is empty and whether its type matches the selected output type. Use **To String** to print it or **Get Script Struct** to check its type. For structs made in Blueprint, the types must match exactly.

## The wildcard pin has the wrong type

Check the picker and any connected struct pin. The connected pin sets the type. Disconnect it before choosing a different type in the picker.

## An array will not connect to a wildcard struct pin

Make, Get and Set accept one struct at a time. Use an array of Instanced Struct values for multiple entries. Use the array nodes to find, filter, count or remove them.

## Parsed text fails after a type was renamed

The exported text includes the struct type. If its name or path changes, add an Unreal Engine redirect from the old type to the new one, or update the type information in the saved text.

## I see similar nodes elsewhere in the menu

Unreal Engine also provides some basic Instanced Struct nodes. This plugin groups its Make, Get, Set, validity, reset and comparison nodes with its additional tools under **Instanced Struct**.

# Support

---

For help or bug reports, contact **[tomasz.klin@gmail.com](mailto:tomasz.klin@gmail.com)**.

Include your Unreal Engine version, plugin version and the struct type involved. A screenshot of the relevant graph is helpful. This manual covers plugin version 1.0.0.